NixOS Installation Guide

Regolith-style i3 desktop · Btrfs · Disko · Home Manager

rajivgangadharan/nixos-config

2026-03-02

Contents

Overview 2

Prerequisites 3

Step 1 — Boot 4

Step 2 — Network 4

Step 3 — Verify disk IDs 4

Step 4 — Get disko 4

Step 5 — Clone the configuration 4

Step 6 — Partition, format and mount 4

Step 7 — Generate hardware configuration 6

Step 8 — Deploy configuration files 6

Step 9 — Add Nix channels 6

Step 10 — Edit configuration.nix 7

Step 11 — Install 7

Step 12 — Reboot 7

Step 13 — Apply dotfiles (Home Manager) 8

Step 14 — Harden passwords (recommended) 8

Snapshot configuration (reference) 10

Key bindings reference 10

Overview

This guide installs NixOS 24.11 from the minimal ISO onto the following hardware, using the declarative configuration at `github.com/rajivgangadharan/nixos-config`.

Disk layout

Drive	Device (by-id)	Role
NVMe 500 GB	`nvme-CT500P3SSD8_2234E65A6AD2`	Root, swap, boot
SATA 500 GB	`ata-CT500MX500SSD1_2013E297EA30`	Home, data

Partition layout

Partition	FS	Mount	Notes
NVMe p1	btrfs `nixos`	various subvolumes	see below
NVMe p2	vfat	`/boot`	EFI System Partition
NVMe p3	swap	—	36 GB, hibernation
SATA p1	btrfs `data`	various subvolumes	`nofail`

Btrfs subvolumes

Subvolume	Mount	Pool
`@`	`/`	NVMe
`@tmp`	`/tmp`	NVMe
`@opt`	`/opt`	NVMe
`@nix`	`/nix`	NVMe
`@var`	`/var`	NVMe
`@snapshots`	`/.snapshots`	NVMe
`@var_snapshots`	`/var/.snapshots`	NVMe
`@home`	`/home`	SATA
`@home_snapshots`	`/home/.snapshots`	SATA
`@data`	`/data`	SATA
`@data_snapshots`	`/data/.snapshots`	SATA

Users

User	sudo	Notes
`rajivg`	yes (wheel)	Password forced to change on first login
`rishir`	no	Password forced to change on first login

Prerequisites

Before booting the USB:

- **Secure Boot disabled** in BIOS/UEFI firmware settings
- **USB boot** first in boot order
- **Ethernet** connected (strongly recommended — avoids WiFi setup during install)
- Confirm both drives are physically present and connected

Download the NixOS 24.11 minimal ISO from `https://nixos.org/download` and write it to a USB drive (replace `/dev/sdX` with your actual USB device):

```
sudo dd if=nixos-minimal-24.11-x86_64-linux.iso \
        of=/dev/sdX bs=4M status=progress
sync
```

Step 1 — Boot

Boot from the USB. NixOS auto-logs in as `nixos` and drops you into a root-capable shell.

Step 2 — Network

Ethernet is usually configured automatically. Verify connectivity:

```
ip a
ping -c 3 nixos.org
```

WiFi (if needed):

```
nmtui
```

Step 3 — Verify disk IDs

The configuration hardcodes serial-number-based device paths. Confirm both drives appear with the expected IDs **before** running disko:

```
ls -l /dev/disk/by-id/ | grep -E 'nvme-CT500|ata-CT500'
```

You must see both entries. If either is missing, or the serial number differs from what is in `disk-config.nix`, stop and update the config first.

Step 4 — Get disko

Disko is not included in the minimal ISO. Drop into a temporary shell to get it:

```
nix-shell -p disko
```

Step 5 — Clone the configuration

```
git clone https://github.com/rajivgangadharan/nixos-config /tmp/cfg
```

Step 6 — Partition, format and mount

> **Warning:** This irreversibly wipes **both** drives.

```
sudo disko --mode zap_create_mount /tmp/cfg/disk-config.nix
```

Disko partitions both drives, formats all filesystems, creates all btrfs subvolumes, and mounts everything under `/mnt`.

Verify all mounts are present:

```
findmnt | grep /mnt
```

Expected output includes all of the following:

```
/mnt
/mnt/boot
/mnt/tmp
/mnt/opt
/mnt/nix
/mnt/var
/mnt/.snapshots
/mnt/var/.snapshots
/mnt/home
/mnt/home/.snapshots
/mnt/data
/mnt/data/.snapshots
```

Step 7 — Generate hardware configuration

> **Critical:** the `--no-filesystems` flag is mandatory. Without it, `nixos-generate-config` writes `fileSystems` entries that conflict with the ones the disko NixOS module generates, causing the build to fail with option-conflict errors.

```
sudo nixos-generate-config --no-filesystems --root /mnt
```

This writes `/mnt/etc/nixos/hardware-configuration.nix` containing your CPU microcode loader, NVMe kernel modules, and other detected hardware configuration.

Step 8 — Deploy configuration files

Copy all files **except** `hardware-configuration.nix` (which you just generated in the previous step — do not overwrite it):

```
sudo cp /tmp/cfg/configuration.nix   /mnt/etc/nixos/
sudo cp /tmp/cfg/disk-config.nix     /mnt/etc/nixos/
sudo mkdir -p /mnt/etc/nixos/home-config
sudo cp /tmp/cfg/home-config/*.nix   /mnt/etc/nixos/home-config/
```

Step 9 — Add Nix channels

Channels must be added as root — they are fetched during `nixos-install`:

```
sudo nix-channel --add \
  https://github.com/nix-community/disko/archive/main.tar.gz \
  disko

sudo nix-channel --add \
  https://github.com/nix-community/home-manager/archive/release-24.11.tar.gz \
  home-manager

sudo nix-channel --update
```

Step 10 — Edit configuration.nix

Open `/mnt/etc/nixos/configuration.nix` in an editor (`nano`, `vi`, etc.) and make the following changes:

a) Uncomment the disko imports (lines 23–24):

```
<disko/module.nix>
./disk-config.nix
```

b) Set your hostname (line 37):

```
networking.hostName = "yourbox";
```

c) Confirm your timezone (line 41):

```
time.timeZone = "America/New_York";
```

Leave the `home-manager` block commented for now. Get a clean first boot before applying dotfiles.

Step 11 — Install

```
sudo nixos-install --root /mnt
```

Near the end of the build you will be prompted to set a **root** password. This is separate from the user accounts. The build takes 10–30 minutes depending on download speed.

If the build fails, fix the error in `/mnt/etc/nixos/` and re-run `nixos-install` — Nix caches all completed work so only the failing step is retried.

Step 12 — Reboot

```
sudo reboot
```

Remove the USB when the machine powers off. The expected first-boot sequence is:

1. systemd-boot menu (auto-boots after 5 seconds)
2. NixOS kernel and services
3. LightDM login screen with Arc-Dark GTK greeter
4. Log in as `rajivg` with password `nixos`
5. Immediately prompted to choose a new password
6. Plain default i3 session — no custom dotfiles yet

Log in as `rishir` as well to trigger the forced password change for that account.

Step 13 — Apply dotfiles (Home Manager)

Log in as `rajivg` and add the Home Manager channel for your user:

```
nix-channel --add \
  https://github.com/nix-community/home-manager/archive/release-24.11.tar.gz \
  home-manager
nix-channel --update
```

Edit `/etc/nixos/configuration.nix` as root and uncomment the Home Manager import and configuration block:

```
# In the imports list:
<home-manager/nixos>

# Near the bottom of the file:
home-manager = {
  useGlobalPkgs   = true;
  useUserPackages = true;
  users.rajivg    = import ./home-config/home-rajivg.nix;
  users.rishir    = import ./home-config/home-rishir.nix;
};
```

Apply the configuration:

```
sudo nixos-rebuild switch
```

Log out and back in. The full Regolith-style i3 desktop is now active for both users:

- Solarized Dark colour scheme
- Arc-Dark GTK theme and icons
- picom compositor with shadows and fading
- dunst notifications
- rofi app launcher (`Super+Space`)
- i3blocks status bar (auto-hide, `Super` to reveal)
- alacritty terminal with MesloLGS Nerd Font

Step 14 — Harden passwords (recommended)

Once both users have set their own passwords, remove the plaintext `initialPassword` values from `/etc/nixos/configuration.nix` and replace them with hashed passwords:

```
# Run for each user and copy the output
mkpasswd -m yescrypt
```

```
users.users.rajivg = {
  # remove:  initialPassword = "nixos";
  # add:
  hashedPassword = "$y$j9T$...";   # output of mkpasswd
};
```

Then rebuild:

```
sudo nixos-rebuild switch
```

Snapshot configuration (reference)

The disk layout is designed for use with `snapper` or `btrbk`. Each snapshotted subvolume has a dedicated sibling subvolume on the **same** btrfs pool (btrfs snapshots cannot cross pool boundaries).

Source	Snapshot subvolume	Mount
`/`	`@snapshots`	`/.snapshots`
`/var`	`@var_snapshots`	`/var/.snapshots`
`/home`	`@home_snapshots`	`/home/.snapshots`
`/data`	`@data_snapshots`	`/data/.snapshots`

To configure snapper after install:

```
sudo snapper -c root  create-config /
sudo snapper -c var   create-config /var
sudo snapper -c home  create-config /home
sudo snapper -c data  create-config /data
```

Key bindings reference

Binding	Action
`Super+Return`	Open terminal (alacritty)
`Super+Space`	App launcher (rofi)
`Super+Shift+Space`	Run command (rofi)
`Super+Ctrl+Space`	Window switcher (rofi)
`Super+Shift+q`	Close focused window
`Super+Shift+e`	Exit i3
`Super+Shift+b`	Reboot
`Super+Shift+p`	Power off
`Super+Shift+s`	Suspend
`Super+Escape`	Lock screen
`Super+i`	Toggle status bar
`Super+Shift+n`	File manager (nautilus)
`Super+c`	Settings (gnome-control-center)
`Super+d`	Display settings
`Super+w`	WiFi settings
`Super+b`	Bluetooth settings
`Print`	Screenshot (saved to `~/Pictures/`)
`Shift+Print`	Screenshot selection
`Super+h/j/k/l`	Focus left/down/up/right
`Super+Shift+h/j/k/l`	Move window
`Super+r`	Resize mode
`Super+f`	Fullscreen toggle
`Super+Shift+f`	Floating toggle
`Super+1...0`	Switch to workspace

Binding	Action
`Super+Shift+1...0`	Move window to workspace